

CO4_AT2_Industry-based Problem

Name: C. Himaja

Reg. No: 19273035

Source code

```
#include <stdio.h>

#define MAX_APPS 20
#define MAX_MEMORY 120

// Structure to store application details
struct Application {
    char name[30];
    int memory;
    int profit;
};

// Function to find maximum profit using 0/1 Knapsack
int knapsack(struct Application apps[], int n, int capacity, int selected[]) {

    int dp[MAX_APPS + 1][MAX_MEMORY + 1];
    int i, w;

    // Initialize DP table
    for (i = 0; i <= n; i++) {
        for (w = 0; w <= capacity; w++) {
            dp[i][w] = 0;
        }
    }
}
```

```

// Build DP table
for (i = 1; i <= n; i++) {
    for (w = 0; w <= capacity; w++) {

        // If application memory requirement exceeds capacity
        if (apps[i - 1].memory > w) {
            dp[i][w] = dp[i - 1][w];
        }
        else {
            // Choose maximum of:
            // 1. Not selecting the application
            // 2. Selecting the application
            int exclude = dp[i - 1][w];

            int include =
                apps[i - 1].profit +
                dp[i - 1][w - apps[i - 1].memory];

            if (include > exclude)
                dp[i][w] = include;
            else
                dp[i][w] = exclude;
        }
    }
}

// Find selected applications by backtracking
w = capacity;

```

```

for (i = n; i > 0; i--) {

    if (dp[i][w] != dp[i - 1][w]) {
        selected[i - 1] = 1;
        w = w - apps[i - 1].memory;
    }
    else {
        selected[i - 1] = 0;
    }
}

return dp[n][capacity];
}

int main() {

    struct Application apps[MAX_APPS] = {
        {"App1", 30, 60},
        {"App2", 20, 50},
        {"App3", 40, 70},
        {"App4", 10, 25},
        {"App5", 50, 80},
        {"App6", 25, 45},
        {"App7", 15, 35},
        {"App8", 35, 65}
    };

    int n = 8;
    int capacity = 120;
    int selected[MAX_APPS] = {0};

```

```

int maxProfit = knapsack(
    apps,
    n,
    capacity,
    selected
);

int totalMemory = 0;

printf("\n=====\\n");
printf(" CLOUD RESOURCE ALLOCATION OPTIMIZATION\\n");
printf("=====\\n");

printf("\nServer Memory Capacity: %d GB\\n", capacity);

printf("\nApplication Details:\\n");
printf("-----\\n");
printf("%-10s %-15s %-10s\\n",
    "Application", "Memory(GB)", "Revenue");
printf("-----\\n");

for (int i = 0; i < n; i++) {
    printf("%-10s %-15d %-10d\\n",
        apps[i].name,
        apps[i].memory,
        apps[i].profit);
}

printf("\nSelected Applications:\\n");

```

```

printf("-----\n");

for (int i = 0; i < n; i++) {
    if (selected[i]) {
        printf("%s -> %d GB -> Revenue %d\n",
            apps[i].name,
            apps[i].memory,
            apps[i].profit);

        totalMemory += apps[i].memory;
    }
}

printf("\n");

printf("\nTotal Memory Allocated = %d GB\n",
    totalMemory);

printf("Unused Memory = %d GB\n",
    capacity - totalMemory);

printf("Maximum Revenue = %d\n",
    maxProfit);

printf("\n=====");

return 0;
}

```